Assignment 1

Univariate and Multivariate Statistics

FinTech 545 — Quantitative Risk Management

Becky Qin

1. Reading the Shape of a Sample

Predict

problem1.csv has one series x, n = 1000 observations. Before I fit anything, I looked at the first four sample moments:

Moment	Value
mean	0.001104
variance	0.00009625
standard deviation	0.009811
skewness	-0.6688
excess kurtosis	2.3398

(a) Which distributions from Week 1 could plausibly have produced this sample?

Table 6 from the Week 1 notes sorts candidate families by the sign of skewness and the sign of excess kurtosis. Here skewness is clearly negative (-0.67, not just noise around 0) and excess kurtosis is clearly positive (2.34), so a few families can be crossed off right away:

- **Normal** — out on both moments. Normal has skewness 0 and excess kurtosis 0, and the kurtosis miss alone (2.34 vs. 0) kills it: no choice of mu or sigma gives a Normal fat tails.
- **Student's t (symmetric)** — out on skewness. A symmetric t is skewness 0 no matter what nu is. It could match the positive kurtosis (something like nu = 6-10 gets you near 2.4), but it just has no knob for skew, so it can't get to -0.67.
- **Symmetric Normal Inverse Gaussian (beta = 0)** — same story as the t: setting beta = 0 pins skewness at 0.
- **Lognormal** — out twice over. It only has support x > 0 and is always positively skewed for sigma > 0. Our data has negative values (the 2nd observation alone is -0.0205) and negative skew, so this one's not even close.

What's left standing is anything that can actually do skew and fat tails at the same time — a skewed NIG, a skewed t, something in the generalized-hyperbolic family. Those are plausible; the four above aren't.

Fit and Reconcile

(b) Fitting a Normal by matching mean and variance.

I fit Normal(mean = 0.001104, sd = 0.009811) — just matching the first two moments, since that's all a Normal can do anyway. Its 1% quantile comes out to -0.02172. In the actual data, 26 out of 1000 observations fall below that number. If the Normal were the right model, we'd expect about 10 (1% of 1000). 26 is roughly 2.6x too many.

(c) What does the fitted Normal get wrong?

The Normal is missing something specifically in the left tail, and it's missing it in one direction: it underestimates how often big losses happen. Checking the right side backs this up — only 6 out of 1000 observations beat the fitted 99th percentile (expected ~10), so the right tail is actually thinner than a

Normal, while the left tail is fatter. That's exactly the fingerprint of negative skew plus positive excess kurtosis: a longer, heavier left tail and a shorter right tail than a symmetric bell curve with the same variance would give you. Practically, if you used this fitted Normal for risk purposes, you'd under-count how often the worst losses happen and over-count how often the biggest gains happen — and both of those errors point the same way (this series is riskier on the downside than the Normal thinks), they don't cancel out.

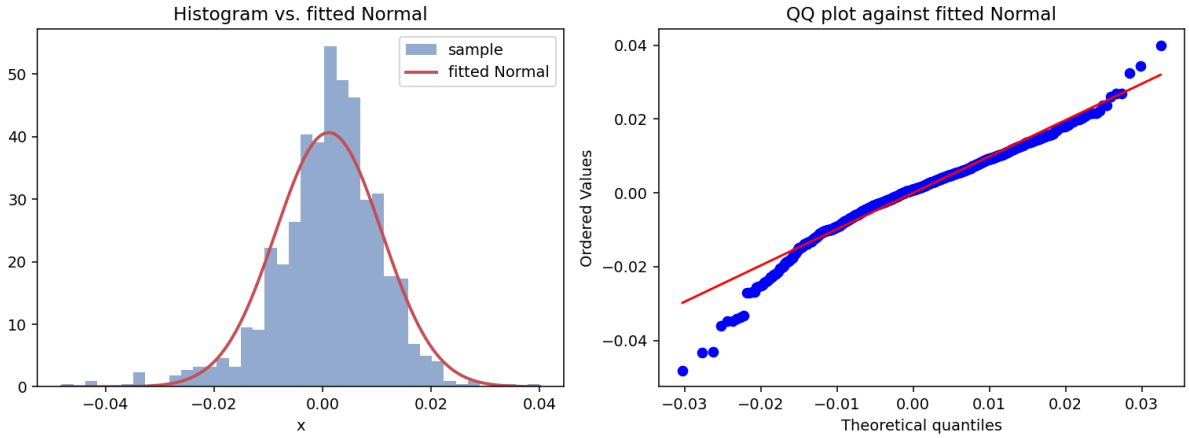


Figure 1: Histogram of the sample against the fitted Normal density (left), and a Normal QQ plot (right). The QQ plot's left tail bends below the 45-degree reference line and the right tail sits close to it, confirming the fatter-left / thinner-right asymmetry.

2. A Regression Whose Errors Are Not Normal

Predict

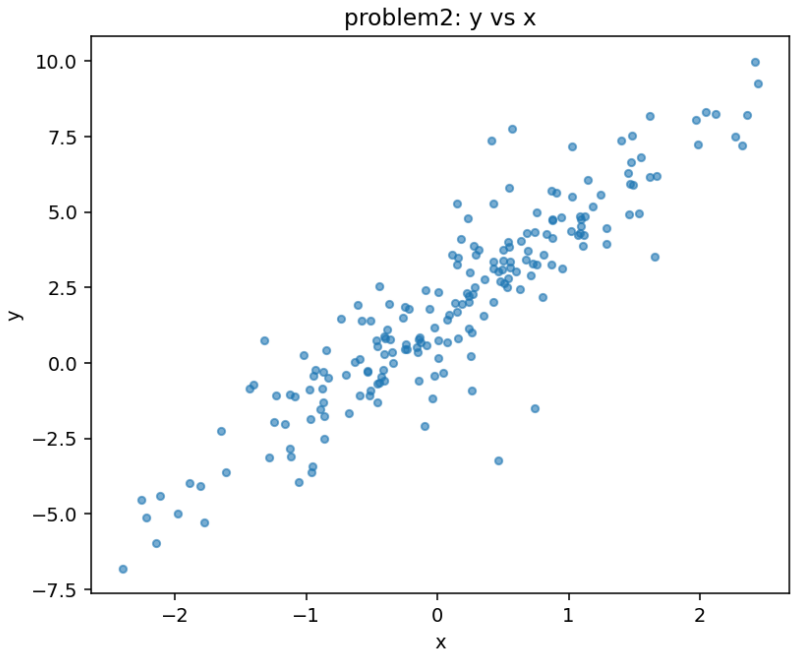


Figure 2: y against x, problem2.csv (n = 200).

(a) What does the scatter show, and what error distribution do I expect?

The scatter is a clean, tight, upward-sloping cloud — there's clearly a linear relationship here (raw correlation is 0.91). What jumps out isn't curvature, it's a handful of points that sit well outside the main band on both ends (bottom-left around x = -2.3, top-right around x = 2.4), while most of the points hug the line pretty closely. A tight core with occasional bigger surprises like that is basically the signature of fat-tailed errors, not Normal ones. So I'd expect a Student's t to fit the residuals better than a Normal — a Normal would keep the spread roughly constant and wouldn't throw out outliers this far out this often.

(b) Which OLS assumption is violated, and what should that do to the slope and its standard error?

Of the seven OLS assumptions (Week 2, Section 3.1), the one being broken here is **assumption 7**, that the errors are normally distributed. I don't think this should move the slope estimate much. OLS's beta-hat is already the best linear unbiased estimator just from assumptions 1-6; normality only matters for making the usual t-tests, confidence intervals, and standard errors exact in finite samples — it's not needed for beta-hat itself to be unbiased or consistent. So my guess: the slope barely moves, but the standard error on it (and anything built on top of it, like p-values or confidence intervals) should be treated with more suspicion than the printed number suggests, since those are calibrated for Normal errors we don't actually have.

Fit

I fit three models on (x, y): OLS with its standard errors, MLE assuming a Normal error, and MLE assuming a Student's t error (fitting alpha, beta, the t scale, and nu all together).

Model	alpha	beta	error params	log-lik	AICc
OLS (with s.e.)	1.5607 (se 0.0963)	2.9908 (se 0.0966)	sigma_hat = 1.3499 (n-2 divisor)	-	-
MLE, Normal error	1.5607	2.9908	sigma = 1.3446 (n divisor)	-343.01	694.23
MLE, t error	1.5480	3.0192	scale = 0.9355, nu = 3.63	-328.56	667.43

Notice the Normal-MLE beta matches the OLS beta exactly to the precision shown, which is exactly what Week 2 section 4.3 says should happen: maximizing a Normal likelihood over beta is the same optimization as minimizing sum of squared errors, just flipped around. The only difference is sigma, since MLE divides by n and OLS's s.e. formula divides by n - p.

Reconcile

(c) Did the violation move the slope as much as I predicted?

Yes, and even less than I expected. The t-model's slope (3.0192) is only about 1% off from the OLS/Normal slope (2.9908), well within one standard error. That lines up with my prediction: changing the error distribution doesn't really move beta-hat, because OLS, Normal-MLE, and t-MLE are all essentially fitting the same central tendency — they just disagree on how much weight to give the tails.

(d) AICc prefers the t model by a wide margin (667.43 vs. 694.23, a gap of nearly 27) even though the slopes agree. What is the rejected (Normal) model getting wrong, if not the slope?

It's getting the *shape* of the error distribution wrong, not the center. The residuals have excess kurtosis around 3.5 — way fatter-tailed than a Normal — plus mild negative skew. A Normal error model has to spread its variance evenly across the whole range just to fit the handful of large residuals, which makes it wrong pretty much everywhere: too wide near zero (predicting more small/medium residuals than we actually see) and too narrow in the tails (predicting way fewer large residuals than actually show up). The t distribution, with basically the same beta, uses its extra degree of freedom (nu = 3.63) to pile probability near the center while still allowing heavy tails, which just matches where the residual mass actually sits. That's what AICc is picking up on — it has nothing to do with where the regression line sits and everything to do with the distribution wrapped around it.

(e) Comparing the 95% and 99.5% quantiles of the two fitted error distributions.

Quantile	Normal error	t error
95%	2.212	2.054
99.5%	3.464	4.620

The Normal is wider at 95%, but the t is way wider at 99.5%. This is basically the standardized-moment story from Week 1 playing out in the tails: since both distributions are fit to residuals with roughly the same overall spread, a fat-tailed distribution has to be narrower than the Normal somewhere to make room for being wider way out in the tail. That extra tail mass in the t has to come from somewhere, and it comes from around the 95% region — which is why the t's 95% quantile (2.05) is actually tighter than the Normal's (2.21). But by 99.5%, the t's heavy tail has completely overtaken the Normal (4.62 vs. 3.46). For

something like a capital buffer, which is meant to survive the rare bad day rather than a typical one, I'd want the t-based quantile. Using the Normal's 99.5% number here would understate the buffer by more than a full unit of y, which is exactly the kind of tail-risk mistake that's the whole reason to fit a fat-tailed error in the first place.

3. Pearson Against Spearman

Predict

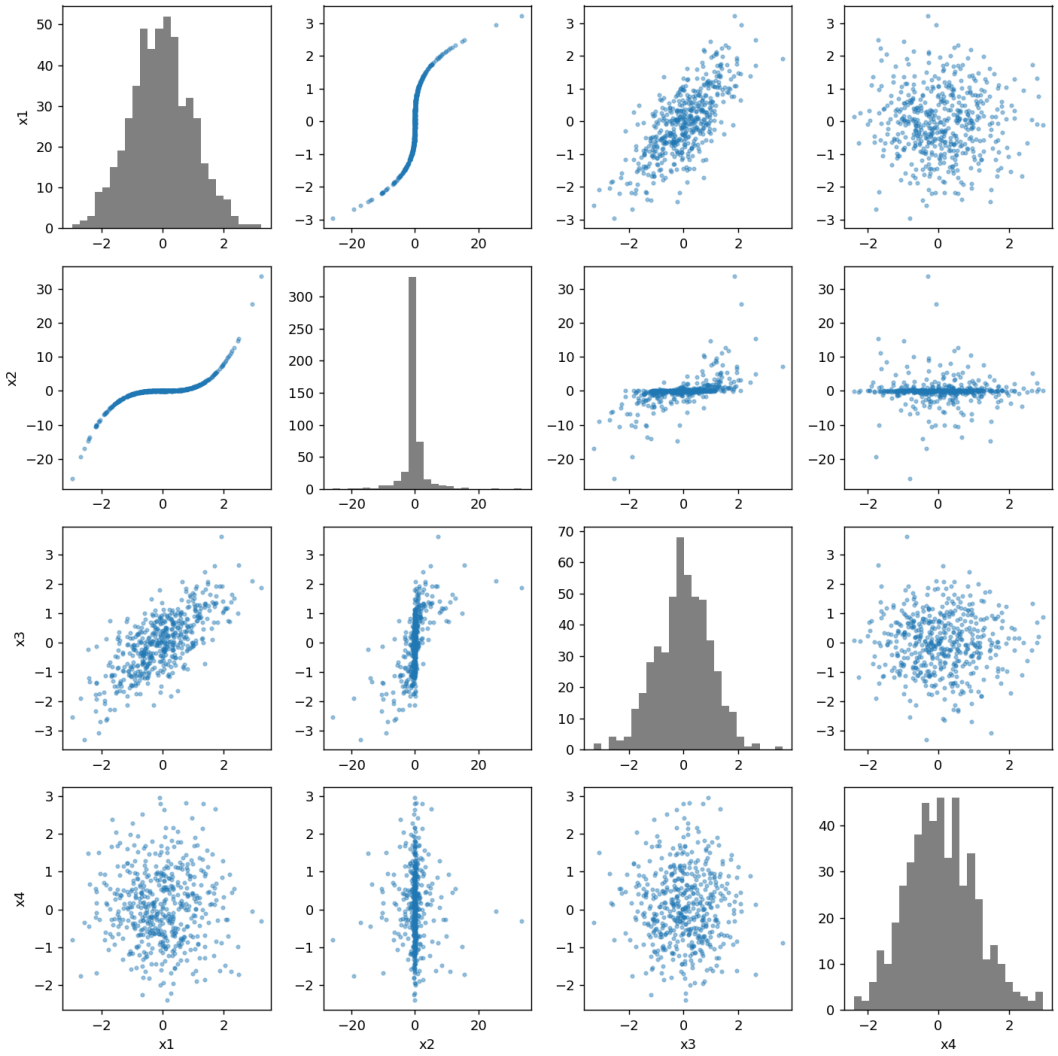


Figure 3: All pairwise scatters (off-diagonal) and marginal histograms (diagonal) for x1-x4.

(a) Which pair should Pearson and Spearman disagree on most, and which should they agree on?

The x1-x2 panel is clearly monotone but strongly curved — looks roughly cubic. Every increase in x1 gives an increase in x2, but the size of that jump grows a lot away from the center. Spearman only cares about ranks, so it should stay close to 1 regardless of the curve, while Pearson (a linear measure) should get pulled down by the curvature and the extreme points it creates. So I'd expect x1 vs. x2 to have the biggest gap. x1 vs. x3 and x2 vs. x3 look like ordinary noisy linear relationships, nothing curved, so I'd expect Pearson and Spearman to line up there. x4 looks uncorrelated with everything, so both should agree there too, near 0.

Fit and Reconcile

(b) Correlation matrices and the largest gap.

Pearson:

	x1	x2	x3	x4
x1	1.000	0.799	0.719	-0.005
x2	0.799	1.000	0.588	-0.021
x3	0.719	0.588	1.000	-0.009
x4	-0.005	-0.021	-0.009	1.000

Spearman:

	x1	x2	x3	x4
x1	1.000	0.972	0.684	0.000
x2	0.972	1.000	0.667	-0.001
x3	0.684	0.667	1.000	-0.002
x4	0.000	-0.001	-0.002	1.000

The largest gap is x1 vs. x2: Pearson = 0.799, Spearman = 0.972, a gap of 0.173 — more than four times the next biggest gap (x2 vs. x3, at 0.080). Matches the prediction exactly.

(c) The mechanism, and which coefficient is honest.

x2 looks like a monotone, convex-then-concave transform of x1 — basically x2 proportional to x1 cubed, the same shape from the Week 2 notes example. A cubic transform keeps the order perfectly (bigger x1 always means bigger x2) but stretches the scale unevenly: small moves in x1 near the center barely move x2, while the same move out in the tails moves x2 a lot. Pearson measures how well a *straight line* fits the pair, so that uneven stretching, plus the leverage from the extreme points it creates, drags Pearson down from what would otherwise be a perfect relationship. Spearman converts both variables to ranks first and then runs Pearson on the ranks; since the transform is monotone, the ranks of x2 basically match the ranks of x1, so Spearman lands close to its max of 1. I'd call Spearman the honest description here: it correctly says that knowing x1 tells you almost everything about the order of x2. Pearson is answering a narrower question — how well does a straight line explain x2 from x1 — and the honest answer to that narrower question really is lower, because a straight line genuinely is a worse fit to a curved relationship, even a perfectly deterministic one.

4. Conditional Distributions

Predict

The sample covariance matrix of (x1, x2) is:

```
Sigma = [[1.09988, 1.69690], [1.69690, 4.10475]]
```

with x1 as block 1 (q = 1) and x2 as block 2 (N - q = 1), following the Week 2, Section 2.4 partition.

(a) Conditional variance formula, and the uncertainty-reduction factor.

With x1 as block 1 and x2 as block 2, the labels in Section 2.4 are flipped from how I'm using them here (there, X1 is the piece being described conditional on X2 = a). Relabeling: the conditional variance of x2 given x1 is

```
Var(x2 | x1) = Sigma_22 - Sigma_21 * Sigma_11^-1 * Sigma_12
```

Since both series are scalars, that's just Sigma_22 - Sigma_12^2 / Sigma_11. Learning x1 shrinks the variance of x2 by

```
factor = Var(x2|x1) / Var(x2) = 1 - Sigma_12^2/(Sigma_11 * Sigma_22) = 1 - rho^2
```

Here Sigma_12^2/(Sigma_11*Sigma_22) = rho^2 = 0.6378 (rho = 0.7986), so the factor is 1 - 0.6378 = 0.3622: knowing x1 shrinks the variance of x2 down to about 36% of what it was (or the sd down to about sqrt(0.362) = 60%). Numerically, Var(x2) = 4.10475 drops to Var(x2|x1) = 1.48675.

(b) Does the factor depend on the observed value of x1?

No. The term Sigma_12 * Sigma_11^-1 * Sigma_21 only involves the covariance structure Sigma — there's no 'a' (the actual observed x1 value) anywhere in that expression. This is just a structural fact about the multivariate normal: the conditional mean moves with whatever you're conditioning on, but the conditional variance is the same number whether x1 happens to be near its mean or way out in a tail. I'm answering this before even touching the data, just from reading which symbol shows up in which formula.

Fit and Reconcile

(c) Conditional mean, in the blocks of Sigma, and its regression interpretation.

```
mu_bar(x2 | x1) = mu_2 + Sigma_12 * Sigma_11^-1 * (x1 - mu_1) = mu_2 + beta * (x1 -
mu_1)
```

The coefficient on x1 is beta = Sigma_12 / Sigma_11 = 1.5428. That's literally the OLS slope you'd get from regressing x2 on x1 (Week 2, Section 3.2: beta-hat = cov(x,y)/var(x) for one regressor) — the conditioning formula and the least-squares formula are the same thing, just arrived at two different ways.

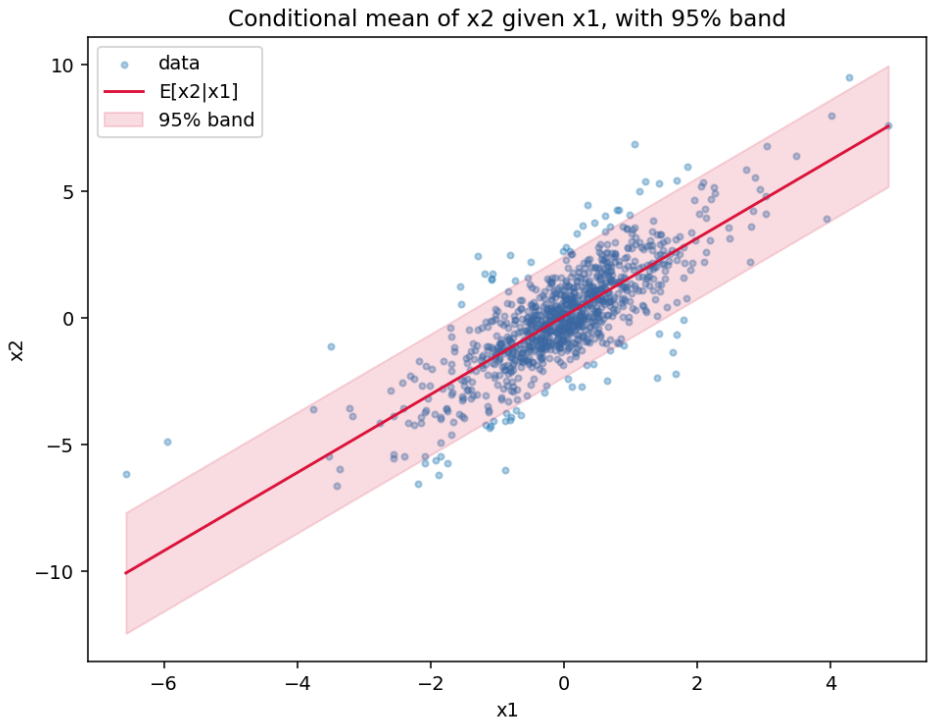


*Figure 4: Scatter of (x1, x2) with the conditional mean E[x2|x1] and a constant-width 95% band (+/-1.96 * sqrt(Var(x2|x1))).*

(d) Fraction of observations inside the band.

93.5% of the 1000 observations land inside the constant-width 95% band — close to the nominal 95%, but noticeably under it.

(e) Coverage by distance from the mean of x1.

Bucket	n	Coverage
|x1 - mean| <= 1 sd	759	95.7%
1 sd < |x1 - mean| <= 2 sd	190	85.8%
|x1 - mean| > 2 sd	51	90.2%

(f) What the uneven coverage tells us, and which assumption fails.

Coverage is best right near the mean of x1 and worst in the 1-2 sd bucket, which isn't the simple story of 'the band is just too narrow everywhere.' Looking at the residual standard deviation by bucket explains it: 1.075 near the mean, 1.576 between 1-2 sd out, 1.580 beyond 2 sd — roughly 50% bigger away from the mean than close to it, versus the single pooled figure of 1.219 the band actually used. So the true conditional variance of x2 given x1 isn't actually constant in x1, the way part (b) said it should be under a multivariate normal — it grows as x1 moves away from its mean. Because the band in part (c) used one pooled sqrt(Var(x2|x1)) everywhere, it ends up a bit too wide near the mean (overcoverage, 95.7%) and clearly too narrow away from it (undercoverage, 85.8% and 90.2%). What's failing is the multivariate-normal assumption from part (b) itself — that Var(x2|x1) doesn't depend on x1 — which tells us (x1, x2) has heavier, x1-dependent dispersion than a bivariate normal, even if the mean structure is still fine. So what survives from part (c): the linear conditional mean and beta are still a good description of the central tendency, since nothing about this failure touches the mean. What doesn't survive: a single

constant-width band being the right way to describe the uncertainty everywhere. A better band would need its width to grow with |x1 - mu_1|, and the conditional variance from part (a) is really more of a sample average than something that applies at every point.

5. Identifying an AR or MA Order

Predict

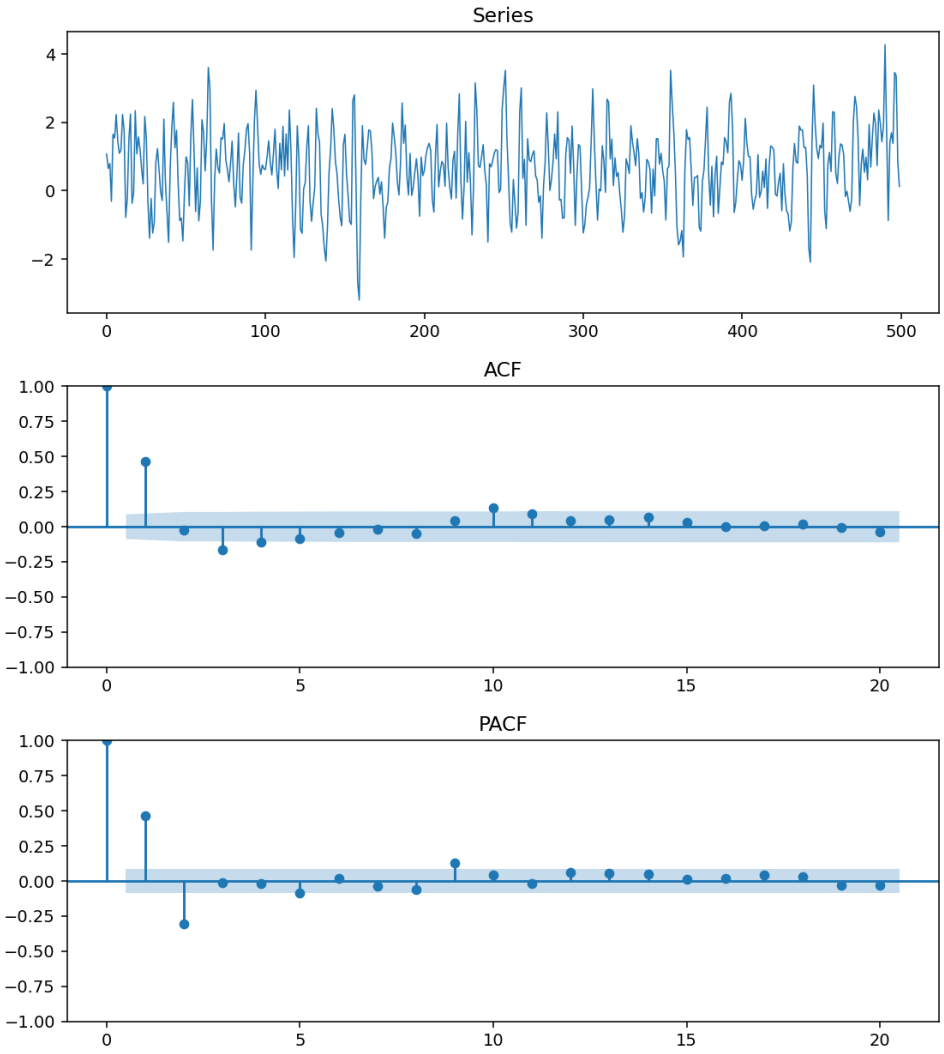


Figure 5: The series (n=500), its ACF, and its PACF, each out to lag 20.

(a) Which function cuts off, and what does that imply?

The PACF has two clearly significant spikes (lag 1 at 0.462, lag 2 at -0.303), then drops inside the significance band and pretty much stays there — it cuts off after lag 2. The ACF doesn't cut off: it starts at 0.462, dips negative around lags 3-4, and just tapers toward zero instead of dropping to (and staying at) zero after some fixed lag — it decays. By Table 3 in the Week 2 notes (ACF decays + PACF cuts off after lag p means AR(p)), this looks like an **autoregressive** process, and the PACF cutting off at lag 2 points to **AR(2)** specifically. That's my guess before fitting anything.

(b) The rule and the significance band.

I used the standard +/-1.96/sqrt(n) band from Week 2, Section 6.1 to call a lag 'significant' (same shaded region as Figures 3-4 in the notes). With n = 500 that's +/-0.0877. PACF lags 1 (0.462) and 2 (-0.303) both clearly clear it; lag 3 onward stays inside (there's one borderline spike at lag 9, but I'd chalk that up

to noise — about 1 in 20 lags is expected to cross the band by chance even when nothing's really there). The ACF, on the other hand, has several lags (1, 3, 4) outside the band without ever really stopping, which is the 'decays' pattern rather than 'cuts off'.

Fit and Reconcile

I fit AR(1)-AR(3) and MA(1)-MA(3) using conditional sum of squares (see the README for why I skipped a canned ARIMA routine), zero-padding any lag that reaches before the start of the series so every model's residual sample is the same length (n = 500) and the AICc numbers below can be compared directly.

Model	Fitted coefficients	AICc
AR(1)	phi1 = 0.462	1418.38
AR(2)	phi1 = 0.602, phi2 = -0.304	1372.09
AR(3)	phi1 = 0.597, phi2 = -0.294, phi3 = -0.017	1373.99
MA(1)	theta1 = 0.529	1388.94
MA(2)	theta1 = 0.633, theta2 = 0.163	1380.90
MA(3)	theta1 = 0.594, theta2 = 0.065, theta3 = -0.156	1373.67

(d) Which model does AICc select, and did it match the prediction?

AICc picks **AR(2)** (1372.09), the lowest number in the table by a comfortable margin over everything except AR(3). That matches my ACF/PACF prediction exactly.

(e) AR(2) vs. AR(3): what AICc trades off, and why R-squared would not make the same choice.

AR(3)'s third-lag coefficient is tiny (-0.017), and its AICc (1373.99) is actually worse than AR(2)'s (1372.09), even though AR(3) has strictly more flexibility to fit the sample. That's AICc doing exactly what it's supposed to: adding a parameter can only reduce (or at worst not change) the sum of squared residuals — the same mechanical point Week 2 Section 5.1 makes about R-squared in a regression, and it applies here too. The extra lag lets the model soak up a bit more of this particular sample's noise, so the in-sample fit (log-likelihood before any penalty) can never get worse by adding it. R-squared has no way to charge for that extra flexibility, so it would just always prefer the bigger model, or be indifferent at best. AICc charges 2k for every extra parameter (plus a small-sample correction), so a lag only helps if it buys back more than 2 units of AIC in log-likelihood. The third lag here barely earns its keep, and AICc correctly calls it overfitting rather than real structure — which matches how small that coefficient is.

A Note on Method (Problem 5)

Quick note on how I actually fit these. For AR(1)-AR(3) I used conditional sum of squares: the AR coefficients minimize the sum of squared one-step residuals, treating any lag before the start of the series as 0. For MA(1)-MA(3) I did the same thing, recursively rebuilding each innovation e_t = y_t - mu - sum(theta_i * e_{t-i}) with e_{t-i} := 0 whenever t - i < 1, and minimizing the sum of squared e_t. For AICc, I counted the fitted coefficients plus the process mean plus the innovation variance as k, which matches how the course library does it elsewhere (library/fitted_model.jl's aicc function: k = number of distribution parameters, plus regression coefficients if there are any). I didn't use StateSpaceModels.jl's SARIMA, even though the professor's own Week 2 demo script does — I couldn't verify that its Kalman-filter likelihood exposes the same parameter count as this course's aicc/FittedModel convention without actually running it myself. CSS is a standard, well-documented way to fit AR/MA models, it's simple enough that I can walk through every line, and since every model here is fit on the same n = 500 zero-padded residual series, the six AICc numbers are on equal footing with each other — which is really all that matters for the comparison this problem is asking for.